INFRAAUDIT ? MANUAL TÉCNICO

Versión 1.0.0 • Auditorías read-only de infraestructura Linux, Docker y red

---

1. Introducción

InfraAudit es una herramienta CLI reutilizable para realizar auditorías tØcnicas y health checks de infraestructura. EstÆ orientada a equipos de consultoría, DevOps y SRE que necesitan evidencia reproducible, hallazgos priorizados por severidad y reportes entregables al cliente.

La herramienta no ejecuta acciones destructivas ni de remediación. Su alcance es exclusivamente de lectura y diagnóstico.

2. Objetivos

- Inventariar y evaluar hosts Linux de forma remota vía SSH
- Auditar runtime Docker (estado, seguridad, recursos)
- Validar servicios de red (DNS, TCP, HTTP/TLS) desde la estación del auditor
- Producir hallazgos estructurados (Finding) con evidencia y recomendaciones
- Exportar resultados a JSON, Markdown y HTML

3. Arquitectura

Componentes principales:

1. CLI (Typer) ? validate, run, report, list-checks
2. Config ? inventario YAML + expansión de variables de entorno + Pydantic
3. Engine ? orquesta checks por categoría y host
4. SSH Client ? Paramiko con timeouts y bloqueo de comandos peligrosos
5. Check plugins ? subclases de BaseCheck registradas en CheckRegistry
6. Reporting ? JSON / Markdown / HTML

Flujo:

‘‘‘
Inventario YAML
  ? Validación
  ? Conexión SSH por host (linux/docker)
  ? Checks de red locales
  ? Agregación AuditReport

? Exporters
‘‘‘

4. Modelo de datos

Severidades

Severidad	Uso típico
INFO	Inventario / estado saludable
LOW	Mejora recomendada
MEDIUM	Riesgo moderado o degradación
HIGH	Problema importante a remediar pronto
CRITICAL	Riesgo inmediato o indisponibilidad

Finding

Cada check produce cero o mÆs hallazgos con: severity, check, target,
message, recommendation, evidence.

5. Checks incluidos

Linux

hostname, distribution, kernel, uptime, CPU, memory, load average, filesystems,
inode usage, mounts, gateway, DNS resolvers, NTP, listening ports, systemd failed units.

Docker

version, containers (running/stopped/health), resource usage, restart policy,
privileged containers, containers as root, healthchecks, networks, volumes.

Network

DNS records, TCP port connectivity, HTTP/HTTPS status + latency, TLS expiry,
issuer, SAN.

6. Inventario YAML

El inventario declara hosts, servicios HTTP/TCP, registros DNS y umbrales.
Las credenciales se inyectan con ${VAR} o ${VAR:-default}.

Nunca incluir passwords en claro en repositorios.

7. Uso de la CLI

‘‘‘bash
pip install -e .
infraaudit validate examples/inventory.example.yml
infraaudit run examples/inventory.example.yml --only network -o results.json
infraaudit report results.json --format html -o report.html
‘‘‘

Filtros:

- --only linux|docker|network
- skip_checks por host en el YAML

8. Extensión
---

1. Crear clase que herede BaseCheck
2. Implementar execute(ctx) -> list[Finding]
3. Registrar en ALL_LINUX_CHECKS / ALL_DOCKER_CHECKS / ALL_NETWORK_CHECKS
4. Aæadir tests con SSH fake / monkeypatch

9. Seguridad operativa

- Preferir autenticación por clave + agent
- Verificar fingerprints y registrar hosts en known_hosts; SSH rechaza claves desconocidas
- Limitar el usuario SSH a permisos de lectura necesarios (sudo sin password solo si es imprescindible y acotado)
- Revisar deny-list de ssh.py antes de aæadir comandos nuevos
- Conservar results.json como evidencia de auditoría

10. Empaquetado y calidad

- Python 3.12+
- pytest para unit tests
- ruff para lint
- mypy para tipos
- Dockerfile + docker-compose.yml para ejecución reproducible

11. Limitaciones conocidas

- Checks DNS CNAME/MX/TXT requieren dig en el PATH del auditor
- Uso de CPU vía /proc/stat es una aproximación de un snapshot
- No reemplaza un vulnerability scanner ni un CSPM cloud
- Ver review.md para la evaluación y límites de la beta

12. Licencia

MIT.

13. Estados de ejecución y cobertura

Los reportes distinguen completed, not_evaluated, error y skipped.
Una comprobación completada puede detectar hallazgos graves. El porcentaje de
cobertura mide checks completados sobre seleccionados, excluyendo omisiones
explícitas. Una auditoría sin trabajo o con checks no evaluables es incompleta
y la CLI devuelve 1 tras guardar evidencia. Ver
mejora 01 para compatibilidad y pruebas.

14. Seguridad y comprobaciones inconclusas
---

assessment separa diagnóstico aprobado, fallo confirmado y resultado inconcluso.
Finding.kind distingue seguridad, operación y hallazgos históricos sin clasificar.
Los errores del auditor no generan vulnerabilidades HIGH/CRITICAL. La evidencia
parcial confirmada se conserva incluso si falta completar otra comprobación. Ver
mejora 02 para semÆntica, reportes y pruebas.

15. Configuración segura y transporte SSH

Las mejoras03 y04 protegen credenciales en errores/reportes y validan el esquema
del inventario antes de conectar. SSH utiliza un plazo total por comando y un
límite combinado de captura stdout/stderr mediante max_output_bytes. Una salida
truncada o timeout produce diagnóstico inconcluso. Ver los documentos específicos
MEJORA-03-PROTECCION-DE-SECRETOS.md, MEJORA-04-VALIDACION-ESTRICTA.md y
MEJORA-02-TRANSPORTE-SSH.md.

ARQUITECTURA DE INFRAAUDIT
==========

Visión

InfraAudit es una CLI de auditoría read-only orientada a consultoría, DevOps y SRE.
Consume un inventario YAML, ejecuta checks no destructivos (SSH + red local) y produce
hallazgos (Finding) exportables a JSON, Markdown y HTML.

Diagrama lógico

```text
??????????????     ???????????????     ??????????????????????
? inventory.yml??????? AuditEngine  ??????? reporters (json/md ?
??????????????     ?            ?     ? html)            ?
               ?  CheckRegistry?     ??????????????????????
               ???????????????
         ????????????????????????????????????
         ?             ?              ?
    Linux checks    Docker checks    Network checks
    (via SSH)       (via SSH)        (local auditor)
```

Componentes

Módulo	Responsabilidad
cli.py	Interfaz Typer (validate, run, report, list-checks)
config.py	Carga YAML, expansión ${ENV}, validación Pydantic
models.py	Finding, CheckResult, HostResult, AuditReport, severidades
ssh.py	Cliente Paramiko con timeouts y deny-list de comandos destructivos
engine.py	Orquestación por host/categoría
checks/	Plugins de checks (BaseCheck + registry)
reporting/	Serialización a JSON/Markdown/HTML

Modelo de Finding

```python
Finding(
  severity,      # INFO | LOW | MEDIUM | HIGH | CRITICAL
  check,         # id del check, p.ej. linux.memory
  target,        # host, servicio o recurso
  message,       # descripción humana
  recommendation,# acción sugerida
  evidence,      # dict con datos crudos
)
```

Extensibilidad

1. Crear una subclase de BaseCheck.
2. Implementar execute(ctx) -> list[Finding].
3. Registrar la instancia en ALL__CHECKS o en un registry custom.
4. El engine la descubrirÆ automÆticamente vía build_default_registry().

Ver creating-checks.md.

Garantías read-only

- No hay comandos de remediación, restart, delete ni write.
- SSHClient bloquea patrones peligrosos (rm, docker rm, systemctl restart, etc.).
- Los checks Docker usan ps, inspect, stats, network ls, volume ls.
- Las credenciales se inyectan por variables de entorno / claves; nunca se hardcodean.

Flujo de ejecución

1. validate ? valida esquema sin conectar.
2. run ? conecta SSH por host, ejecuta checks filtrados (--only), corre checks de red.
3. Persiste results.json.
4. report ? regenera Markdown/HTML desde el JSON.

Timeouts y errores

- Timeout SSH configurable por host (timeout) y global (default_timeout).
- HTTP/TCP con timeouts propios.
- Fallos de check se convierten en Finding HIGH con evidencia del error; no abortan toda la corrida.
- Fallo de conexión SSH se registra en HostResult.connection_error.

Decisiones de diseæo

1. Pydantic v2 para contratos claros y mensajes de validación œtiles en consultoría.
2. Plugins por herencia (BaseCheck) en lugar de entry-points complejos: simple de auditar y enseæar.
3. Paramiko síncrono para portabilidad y control fino de timeouts (asyncssh queda como evolución).
4. Severidades fijas alineadas a prÆcticas de riesgo (INFO?CRITICAL) para priorizar remediación.

5. Reportes autocontenidos (HTML sin CDN) para entregar al cliente offline.
6. Separación network vs host: checks de red salen desde la estación del auditor;
Linux/Docker vía SSH.

GU˝A: CREAR UN NUEVO CHECK
==========

Contrato

Todo check hereda de BaseCheck e implementa execute:

‘‘‘python
from infraaudit.checks.base import BaseCheck, CheckContext
from infraaudit.models import CheckCategory, Finding, Severity

class SwapCheck(BaseCheck):
    check_id = "linux.swap"
    category = CheckCategory.LINUX
    description = "Evalœa uso de swap"
    requires_ssh = True
    requires_docker = False

    def execute(self, ctx: CheckContext) -> list[Finding]:
        assert ctx.ssh is not None and ctx.host is not None
        result = ctx.ssh.run("cat /proc/swaps")
        lines = [ln for ln in result.stdout.splitlines()[1:] if ln.strip()]
        if not lines:
            return [
                self.finding(
                    severity=Severity.LOW,
                    target=ctx.host.id,
                    message="Sin swap configurado",
                    recommendation="Evaluar si el workload requiere swap",
                    evidence={"raw": result.stdout},
)
            ]
        return [
            self.finding(
                severity=Severity.INFO,
                target=ctx.host.id,
                message=f"Swap entries: {len(lines)}",
                evidence={"raw": result.stdout.strip()},
)
        ]
‘‘‘

Registro

Aæadir la instancia a la lista correspondiente:

‘‘‘python
SRC/INFRAAUDIT/CHECKS/LINUX/__INIT__.PY
==========
ALL_LINUX_CHECKS = [
    ...,
    SwapCheck(),
]
‘‘‘

O registrar en runtime:

‘‘‘python
from infraaudit.checks import build_default_registry
registry = build_default_registry()
registry.register(SwapCheck())
‘‘‘

Buenas prÆcticas
---

1. Solo lectura: nunca modificar estado remoto.
2. Evidencia œtil: incluir salida parseada y cruda cuando aporte.
3. Recomendación accionable: quØ hacer, no solo quØ estÆ mal.
4. Severidad coherente con umbrales de ctx.thresholds cuando aplique.
5. Timeouts: confiar en ctx.ssh.run / timeouts del inventario.
6. Tests: mockear SSHClient.run con CommandResult.
7. IDs estables: categoria.nombre (ej. linux.swap, docker.security).

Checks de red (sin SSH)

‘‘‘python
class MyNetCheck(BaseCheck):
    check_id = "network.custom"
    category = CheckCategory.NETWORK
    requires_ssh = False

    def execute(self, ctx: CheckContext) -> list[Finding]:
        service = ctx.extras["my_service"]
        ...
‘‘‘

El engine debe pasar el objeto en extras (ver AuditEngine._run_network_checks).

Categorías y filtros CLI

```bash
infraaudit run inventory.yml --only linux
infraaudit run inventory.yml --only docker,network
infraaudit list-checks
```

Por host se puede omitir checks:

```yaml
hosts:
  - id: web-01
    skip_checks: ["linux.mounts", "docker.volumes"]
```

Evidencia insuficiente

Usar ctx.command(comando) para verificar código de salida y datos. Solo usar allow_empty=True cuando una salida vacía exitosa tenga significado válido. Lanzar CheckNotEvaluableError(motivo, evidence=datos) ante falta de herramientas, permisos o evidencia; nunca sustituir datos desconocidos por cero o estado saludable. Un resultado vacío del plugin se considera no evaluable. Ver la mejora 01.

Clasificación de seguridad

Declarar finding_kind = FindingKind.SECURITY en checks que evalúan controles de seguridad, o pasar kind a self.finding. Un error técnico no es un hallazgo de seguridad. Conservar hallazgos parciales confirmados mediante el argumento findings de CheckNotEvaluableError. Ver mejora 02.

INSTALACIÓN Y EJECUCIÓN
==========

Requisitos

- Python 3.12+
- Acceso SSH a los hosts auditados (clave o password vía env)

- Opcional: Docker Engine en hosts con docker: true
- Opcional: dig en la estación del auditor para DNS no-A/AAAA

Instalación local

‘‘‘bash
git clone https://github.com/alejandrogarcialamazon/infraaudit.git
cd infraaudit
python3.12 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -e ".[dev]"
‘‘‘

Verificar:

‘‘‘bash
infraaudit version
infraaudit list-checks
infraaudit validate examples/inventory.example.yml
‘‘‘

Variables de entorno

Variable	Uso
INFRAAUDIT_SSH_USER	Usuario SSH
INFRAAUDIT_SSH_KEY	Ruta a clave privada
INFRAAUDIT_SSH_PASSWORD	Password (alternativa)
INFRAAUDIT_HOST_WEB	Host de ejemplo web
INFRAAUDIT_HOST_DB	Host de ejemplo DB
INFRAAUDIT_HTTP_URL	URL HTTP de ejemplo

Soporta interpolación ${VAR} y ${VAR:-default} en el YAML. Una variable requerida sin definir impide ejecutar el inventario. El archivo .env no se carga automÆticamente; exportar las variables antes de ejecutar.

SSH requiere la clave del servidor en ~/.ssh/known_hosts. Verificar el fingerprint por un canal confiable antes de registrar el host. Las claves desconocidas o cambiadas se rechazan. En Docker, montar tambiØn el archivo known_hosts junto con las claves.

Ejecución

‘‘‘bash

VALIDAR INVENTARIO
==========
infraaudit validate examples/inventory.example.yml

AUDITOR˝A COMPLETA
==========
infraaudit run examples/inventory.example.yml -o results.json

SOLO DOCKER
==========
infraaudit run examples/inventory.example.yml --only docker

SOLO RED (DESDE LA ESTACIÓN LOCAL)
==========
infraaudit run examples/inventory.example.yml --only network

REPORTES
==========
infraaudit report results.json --format markdown -o report.md
infraaudit report results.json --format html -o report.html
‘‘‘

Códigos de salida:

Code	Significado
0	Auditoría completa sin CRITICAL; puede contener HIGH/MEDIUM/LOW
1	Inventario invÆlido, conexión SSH fallida o check con error
2	Hallazgos CRITICAL o argumentos CLI invÆlidos (consultar stderr)

Docker

‘‘‘bash
docker compose build
docker compose run --rm infraaudit validate /work/examples/inventory.example.yml

AUDITOR˝A (MONTAR SSH KEYS Y USAR NETWORK_MODE: HOST SI HACE FALTA)
==========
docker compose run --rm \
  -v "$HOME/.ssh:/home/infraaudit/.ssh:ro" \
  infraaudit run /work/examples/inventory.example.yml -o /work/out/results.json
‘‘‘

Desarrollo

‘‘‘bash

pytest
ruff check src tests
mypy src
‘‘‘

Seguridad

- No almacenar passwords en el YAML en claro sin env vars.
- Preferir claves SSH con passphrase y ssh-agent.
- Ejecutar siempre desde una estación de confianza (la red se chequea localmente).
- Revisar skip_checks y --only antes de entregas a clientes.